Практикум студента

Автоматические проверки перед выпуском

Цель: обнаружить неверную настройку, исправить её, пройти проверки и выпустить архив конкретного коммита.

После объяснения выполните работу самостоятельно за 60 минут. Используйте отдельную распакованную папку этой пары. Все команды вводятся в Git Bash по одной строке из корня стенда. Символ $ как приглашение терминала не вводится; в команде echo $? он является необходимой частью выражения.

Подготовка Windows 10 и 11

1. Скачайте 03_Стартовый_проект.zip. В Проводнике нажмите «Извлечь всё». Выберите новую папку в Документах и откройте внутри git-ops-p1. Не работайте внутри ZIP и не распаковывайте поверх старой попытки.

2. Дважды щёлкните 00_OPEN_GIT_BASH.cmd. Если помощник не нашёл Git, нажмите правой кнопкой по свободному месту папки и выберите Open Git Bash here. В Windows 11 может потребоваться «Показать дополнительные параметры». Не запускайте от администратора.

3. Если Git for Windows уже установлен для предыдущих работ, переустановка не нужна. Официальная страница установки: https://git-scm.com/install/windows. После установки заново откройте Git Bash. env.sh проверяет наличие git, bash, tar, sha256sum и mktemp. Python, сервер и дополнительные пакеты для задания не нужны.

4. Если установка недоступна, сообщите преподавателю. Пока можно заполнить прогнозы и объяснения по примерам, но техническую часть необходимо завершить позже. После подготовки среды практика работает без интернета.

Папки проекта

baseline.bundle содержит готовую исходную историю. setup.sh создаёт work. evidence хранит протоколы, artifacts — архивы выпуска, REPORT.txt — ваш отчёт, submission — подготовленную сдачу. Папку inspect вы создадите для проверки распакованного выпуска.

Правила работы в терминале

Текущая папка во всех шагах — git-ops-p1. Не выполняйте cd work. Команды git -C work сами обращаются к нужному репозиторию, сохраняя текущую папку терминала.

Каждую строку копируйте отдельно и нажимайте Enter. Для вставки в Git Bash используйте Shift+Insert или меню правой кнопки. Кавычки должны быть прямыми. Команды не вводятся в PowerShell, cmd, Блокнот или окно Python.

Сохранение сообщений

Конструкция > file 2>&1 сохраняет обычный вывод и сообщения ошибок в указанный файл. Команда echo $? сохраняет код непосредственно предшествующей команды. Не вставляйте между ними cat, pwd или другую команду. В нужном шаге эта последовательность приведена полностью.

printf с \n создаёт строки маленького учебного файла. Символ > заменяет этот файл целиком, >> добавляет строку в конец. В командах ниже это намеренно применяется только к config.txt и README.txt подготовленного проекта.

Просмотр файлов в Windows

REPORT.txt открывайте в Блокноте и сохраняйте в UTF-8. Для .sh выбирайте «Открыть с помощью» → Блокнот, если хотите только прочитать код. Выполнять .sh нужно через bash в Git Bash. Не изменяйте проверки ради получения PASS.

Расширения имён: Windows 10 → Проводник → «Вид» → «Расширения имён файлов»; Windows 11 → «Вид» → «Показать» → «Расширения имён файлов». Отчёт должен называться REPORT.txt, а не REPORT.txt.txt.

Помощь при ошибке

No such file: выполните pwd и ls, проверьте распаковку и текущую папку. Missing command: убедитесь, что открыли Git Bash из Git for Windows. Незакрытая кавычка: Ctrl+C и повторите одну строку. Если открылся просмотр истории, q закрывает просмотрщик; в практикуме использован --no-pager.

Для новой попытки распакуйте новый ZIP в другую папку. Не удаляйте .git, не выполняйте reset --hard и не подгоняйте протоколы вручную. Для обращения к преподавателю сохраните номер шага, команду, pwd и полный вывод.

A1 Подготовка стенда

Начальная папка git-ops-p1. Выполняйте строки по порядку.

```
pwd
bash env.sh
bash setup.sh
git -C work config user.name "Student Name"
git -C work status -sb
```

Ожидаемый результат

Создан work на ветке main. Замените Student Name своим именем. Все следующие команды выполняются из корня стенда, без cd work.

Самостоятельное объяснение

Включите расширения имён файлов. Откройте work/config.txt, README.txt, check.sh и tests.sh в Блокноте, ничего не меняя. Найдите допустимые значения и объясните роль каждого файла.

Если результат отличается

Если work уже существует, setup останавливается. Продолжайте текущую попытку либо извлеките новый ZIP в другую папку.

A2 Неверное изменение и отказ проверки

Начальная папка git-ops-p1. Выполняйте строки по порядку.

```
printf 'interval=0\nretries=3\n' > work/config.txt
bash ci.sh verify > evidence/01_failed.txt 2>&1
echo $? > evidence/01_exit.txt
cat evidence/01_exit.txt
cat evidence/01_failed.txt
```

Ожидаемый результат

Код 1. FAIL: config must contain interval=30 or 60 and retries=3. Выпуск не создаётся. После ci сразу выполняйте echo $?, иначе запишете код другой команды.

Самостоятельное объяснение

Прочитайте первый протокол и объясните, почему выпуск должен остановиться. Запишите код 1 и точное сообщение в отчёт.

Если результат отличается

Это ожидаемый отказ. Если код 0, проверьте, действительно ли config содержит interval=0. Не исправляйте протокол вручную: повторите ci и сразу echo.

A3 Исправление конфигурации и описания

Начальная папка git-ops-p1. Выполняйте строки по порядку.

```
printf 'interval=30\nretries=3\n' > work/config.txt
printf 'Service configuration\n' > work/README.txt
printf 'Monitoring interval: 30 seconds.\n' >> work/README.txt
git -C work --no-pager diff -- config.txt README.txt
bash ci.sh verify > evidence/02_pass.txt 2>&1
cat evidence/02_pass.txt
```

Ожидаемый результат

PASS для 30, четыре TEST PASS и VERIFY PASS. Изменены только config.txt и README.txt.

Самостоятельное объяснение

Объясните все четыре теста. Для тестов 3 и 4 успех означает ожидаемый отказ проверки. Найдите эти условия в tests.sh, не редактируя его.

Если результат отличается

Если README and config disagree, сравните число 30 в обоих файлах. Если TEST FAIL, не редактируйте tests.sh ради успеха; сообщите ошибку преподавателю.

A4 Сохранение проверенной версии

Начальная папка git-ops-p1. Выполняйте строки по порядку.

```
git -C work add config.txt README.txt
git -C work commit -m "Set verified monitoring interval to 30"
git -C work status -sb
bash ci.sh release > evidence/03_release.txt 2>&1
cat evidence/03_release.txt
cat artifacts/manifest.txt
```

Ожидаемый результат

Два коммита в main, чистое рабочее дерево. RELEASE PASS и три файла в artifacts. manifest указывает на HEAD.

Самостоятельное объяснение

Сравните список изменённых файлов со своей правкой. Найдите хеш в manifest и проверьте его связь с историей.

Если результат отличается

Commit changes before release: изменения ещё не сохранены коммитом. Artifacts already exist: выпуск уже создан; не запускайте release повторно для продвижения дальше.

A5 Проверка выпуска

Начальная папка git-ops-p1. Выполняйте строки по порядку.

```
git -C work rev-parse HEAD
cat artifacts/manifest.txt
(cd artifacts && sha256sum -c release.sha256)
mkdir inspect
tar -xf artifacts/release.tar -C inspect
bash inspect/check.sh inspect/config.txt inspect/README.txt
ls -a inspect
```

Ожидаемый результат

release.tar: OK. В inspect проверка PASS для 30. В архиве есть файлы версии, но нет каталога .git.

Самостоятельное объяснение

Объясните, что подтверждает SHA256 OK, и перечислите ограничения проверки. Посмотрите содержимое inspect: объясните, почему там нет .git.

Если результат отличается

inspect должна быть новой папкой. Если она уже существует, выберите inspect2 и замените имя во всех командах этого шага. SHA256 FAILED: остановитесь, не создавайте новую контрольную сумму для повреждённого файла.

A6 Отчёт и сдача

Откройте REPORT.txt в корне стенда через Блокнот. Замените каждую пометку [заполнить] собственным ответом. Сохраните файл в UTF-8. Укажите незавершённые этапы и ошибки честно. Автоматическая проверка заполнения не оценивает смысл текста.

В Git Bash из корня стенда выполните:

```
bash pack.sh
```

Ожидается READY. Помощник создаст submission с report.txt, work.bundle, history.txt, папкой evidence и release.tar, release.sha256, manifest.txt.

В Проводнике откройте submission и выделите её содержимое, Ctrl+A. Windows 10: правая кнопка → «Отправить» → «Сжатая ZIP-папка». Windows 11: «Сжать в ZIP-файл» либо соответствующий пункт через дополнительные параметры. Назовите архив GitOps_pair1_result.zip и перенесите его рядом с submission, чтобы повторная упаковка не включила старый ZIP.

Откройте ZIP и проверьте, что внутри есть report.txt, work.bundle и evidence. Сохраните ZIP и исходную рабочую папку до проверки. Позже загрузите один ZIP в соответствующее задание ЭОС после объявления ссылки и срока. Если есть отдельная кнопка окончательной отправки, нажмите её.

Критерии оценивания

Получен и сохранён ожидаемый отказ с кодом 1	2
Исправлены config и README, четыре теста пройдены	2
Проверенные изменения сохранены коммитом, выпуск связан с HEAD	2
Проверена сумма SHA256 и содержимое распакованного выпуска	2
Отчёт объясняет границы проверок, подготовлена сдача	2

Каждый критерий: 2 — выполнено верно и объяснено; 1 — частично либо объяснение неполное; 0 — отсутствует или неверно. Максимум 10 баллов.